



TÉCNICO SUPERIOR EN DESARROLLO DE
APLICACIONES WEB

DEPARTAMENTO DE INFORMÁTICA

PROYECTO

VOLTIO



MANUAL DE TÉCNICO

AUTOR: Alejandro Rey Tostado
Curso Académico: 2025-2026

ÍNDICE

1. Introducción	3
2. Arquitectura de la aplicación	4
2.1. Frontend	4
2.1.1. Tecnologías usadas	4
2.1.2. Justificación de la elección	4
2.1.3. Entorno de desarrollo	5
2.2. Backend	5
2.2.1. Tecnologías usadas	5
2.2.2. Justificación	5
2.2.3. Entorno de desarrollo	5
3. Documentación técnica	6
3.1. Análisis	6
3.1.1. Roles principales	6
3.1.2. Requisitos funcionales	6
3.1.3. Requisitos no funcionales	7
3.2. Desarrollo	7
3.2.1. Base de Datos	7
3.2.2. Justificación del diseño	8
3.2.3. Diagrama de casos de uso	8
3.2.4. Pruebas realizadas	8
4. Proceso de despliegue	8
4.1. Software necesario	8
4.2. Pasos para desplegar la aplicación	8
5. Propuestas de mejoras	9
6. Bibliografía	9

1. Introducción

Voltio es una aplicación web desarrollada como proyecto final del ciclo de Técnico Superior en Desarrollo de Aplicaciones Web. Su finalidad es ofrecer a las empresas del sector eléctrico una plataforma centralizada para el control total de sus operarios, la asignación de tareas en obras y la gestión del inventario de materiales, con una experiencia de uso sencilla y accesible desde cualquier dispositivo.

La aplicación sigue una arquitectura front-only: toda la lógica de presentación y de interacción reside en el cliente (React + Vite + Tailwind CSS) y los servicios de autenticación, base de datos y almacenamiento de ficheros se delegan en Supabase, que actúa como backend-as-a-service (BaaS). De este modo se simplifica la infraestructura, se reduce el número de servicios a desplegar y se facilita el mantenimiento del proyecto.

A nivel funcional, Voltio permite:

- Registro, inicio, cierre de sesión y gestión de usuarios mediante Supabase Auth.
- Visualización de un panel de control tipo dashboard con métricas de seguimiento y gráficos de carga de trabajo en tiempo real.
- Gestión CRUD de Obras y asignación de Tareas específicas a los operarios de la plantilla.
- Control de inventario y materiales, incluyendo un sistema de carga masiva de stock mediante archivos CSV.
- Un panel de administración avanzado donde el rol de administrador puede listar empleados, modificar sus datos y gestionar los permisos (roles) de la plataforma.
- Sistema multi-idioma (Español/Inglés) e interfaz adaptable con Modo Oscuro/Claro.

La motivación principal del proyecto es cubrir la necesidad de digitalizar y optimizar el rendimiento de los equipos de campo en las instalaciones eléctricas, eliminando el uso de papel y centralizando la información operativa.

2. Arquitectura de la aplicación

2.1. Frontend

2.1.1. Tecnologías usadas

- **Lenguaje principal:** TypeScript con TSX para asegurar el tipado estático y la robustez del código. • Framework / Librerías:
- **React v18+:** Construcción de la SPA mediante componentes funcionales y hooks. - Vite: Herramienta de bundling y desarrollo rápido.
- **React Router Dom:** Gestión de rutas públicas (Landing, Login, Registro) y privadas protegidas por roles.
- **Tailwind CSS & Shadcn UI:** Sistema de utilidades CSS combinado con componentes accesibles para una maquetación profesional y responsive.
- **Zustand:** Sistema de gestión del estado global ligero y rápido, utilizado para manejar la sesión del usuario, los datos del administrador, las obras y el inventario.
- **Recharts:** Librería de renderizado de gráficos utilizada en el Dashboard para mostrar la carga de trabajo por operario.
- **i18next:** Framework de internacionalización para ofrecer la plataforma en múltiples idiomas.
- **Lucide React:** Colección de iconos vectoriales.

2.1.2. Justificación de la elección

- React, Vite y TypeScript permiten un desarrollo ágil, libre de errores de tipado y con un ecosistema maduro.
- Zustand se eligió por encima de Redux o Context API por su simplicidad sintáctica y su capacidad para aislar lógicas de negocio complejas en Stores independientes.
- Tailwind CSS y Shadcn UI aceleran enormemente la creación de interfaces coherentes, modernas y adaptables, facilitando la implementación del modo oscuro nativo.

2.1.3. Entorno de desarrollo

- **Sistema operativo:** Windows / macOS / Linux.
- **IDE:** Visual Studio Code con extensiones para TypeScript, TailwindCSS e integración con Git. • Gestor de paquetes: npm / pnpm.
- **Control de versiones:** Git + GitHub (Repositorio: <https://github.com/reeeeyy05/Voltio>)

2.2. Backend

Al igual que en metodologías modernas Serverless, en este proyecto no se ha desarrollado un backend monolítico propio, optando por el servicio BaaS de Supabase.

2.2.1. Tecnologías usadas

- **Supabase PostgreSQL:** Base de datos relacional para almacenar perfiles, obras, tareas y materiales.
- **Supabase Auth:** Autenticación segura y control de sesiones en el cliente.
- **Supabase Storage:** Almacenamiento optimizado de imágenes (avatares).
- **Row Level Security (RLS):** Políticas de seguridad SQL que garantizan el principio de mínimos privilegios según el rol del usuario ('admin' o 'empleado').

2.2.2. Justificación

Supabase proporciona en un único servicio integrado todo lo necesario para producción, reduciendo drásticamente la configuración de infraestructura y garantizando alta disponibilidad. El uso de PostgreSQL asegura integridad referencial y permite escalar el proyecto en el futuro.

2.2.3. Entorno de desarrollo

- Dashboard web de Supabase para la creación de tablas, triggers y definición de políticas RLS.

- Vercel como plataforma de despliegue del frontend, configurada con las variables de entorno (VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY).

3. Documentación técnica

3.1. Análisis

3.1.1. Roles principales

- **Invitado:** Puede visualizar la Landing Page con las características del sistema y acceder a los formularios de login y registro.
- **Empleado:** Usuario operativo que accede a un panel personalizado con sus obras asignadas, puede actualizar el estado de sus tareas ("Finalizada" o "Pendiente") y consultar/gestionar el inventario.
- **Administrador:** Dispone de privilegios totales. Puede crear obras, visualizar la carga de trabajo global, hacer carga masiva de inventario y gestionar los roles de la plantilla desde el panel de usuarios.

3.1.2. Requisitos funcionales

- **RF1:** Registro, autenticación y subida de avatar fotográfico mediante Supabase Auth y Storage.
- **RF2:** Dashboard diferenciado por rol: gráficos de rendimiento global para el Administrador y panel de progreso personal para el Empleado.
- **RF3:** Gestión de Obras (CRUD) indicando su estado binario ("Pendiente" o "Finalizada").
- **RF4:** Asignación de Tareas dentro de cada obra, vinculando un empleado específico al proyecto.
- **RF5:** Gestión de Inventario con capacidad de añadir materiales uno a uno o mediante carga masiva a través de la importación nativa de archivos CSV.
- **RF6:** Panel de Administración donde el Admin puede promocionar o degradar roles y actualizar los datos

personales de la plantilla.

3.1.3. Requisitos no funcionales

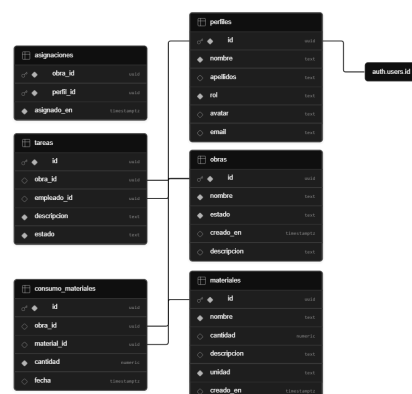
- **RNF 1 - Arquitectura SPA:** Navegación fluida sin recargas de página y redirecciones protegidas.
- **RNF 2 - Seguridad (RLS):** Bloqueo a nivel de servidor de acciones no autorizadas, combinando protección de rutas en React con políticas SQL en Supabase.
- **RNF 3 - Internacionalización:** Soporte nativo para alternar el idioma de la plataforma al instante (i18next).
- **RNF4 - Interfaz y usabilidad:** UI/UX moderna, responsive, con transiciones suaves, notificaciones Toast (Sonner) y Modo Oscuro.

3.2. Desarrollo

3.2.1. Base de Datos

El modelo relacional de Voltio gira en torno a la identidad del usuario y la operativa de obras:

- La tabla perfiles tiene una relación 1:1 directa con el esquema interno auth.users de Supabase.
- La tabla de obras actúa como entidad principal de los proyectos.
- La tabla tareas es una tabla intermedia que relaciona a los empleados (1:N) con las obras (1:N) indicando el trabajo exacto a realizar.
- La tabla de materiales representa el catálogo general de stock.



3.2.2. Justificación del diseño

Al vincular todo directamente con `auth.users`, se garantiza que las políticas RLS de Supabase puedan evaluar de forma inmediata el `auth.uid()` del usuario que realiza la petición web, dotando al sistema de una seguridad infalible contra accesos indebidos.

3.2.3. Diagrama de casos de uso

diagrama

3.2.4. Pruebas realizadas

Se han llevado a cabo pruebas manuales exhaustivas en los diferentes flujos críticos de la plataforma:

- **Rutas protegidas:** Intento de acceso a `/app/usuarios` con una cuenta de rol empleado, resultando en una redirección exitosa bloqueada por el componente `ProtectedRoute`.
- **Sincronización de estado:** Comprobación del ciclo de vida de Zustand al refrescar la página (F5), asegurando que la bandera `isInitialized` retiene la sesión activa y evita parpadeos en el Header.
- **Lógica de Carga Masiva (Inventario):** Subida de ficheros CSV formateados. El módulo `FileReader` procesa correctamente la separación por comas e inyecta iterativamente los materiales en la base de datos.

4. Proceso de despliegue

4.1. Software necesario

- Navegador web moderno.
- Cuenta en Supabase y Vercel.
- Entorno [Node.js](#) instalado correctamente.

4.2. Pasos para desplegar la aplicación

1. Generar la infraestructura en Supabase: creación del proyecto, ejecución del script SQL de las tablas y políticas, y

configuración de los Buckets de Storage.

2. Configurar el archivo .env.local con VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY.
3. Vincular el repositorio de GitHub con un nuevo proyecto en Vercel.
4. Configurar las variables de entorno en el panel de Vercel y ejecutar el despliegue de producción.

5. Propuestas de mejoras

De cara a futuras iteraciones, se contemplan las siguientes integraciones:

- **Módulo de Consumo de Materiales en Obra:** Implementar una vista que relacione los materiales gastados descontándolos directamente del inventario central desde el panel de tareas del operario.
- **Notificaciones Push y Alertas:** Aviso en tiempo real al operario cuando se le asigne una nueva tarea en una obra.
- **Generación de Reportes (jsPDF):** Exportación de balances de finalización de obra o estado del inventario para enviarlos a gerencia o clientes.

6. Bibliografía

- **React & TypeScript:** Documentación oficial y buenas prácticas. (<https://es.react.dev/>) • **Zustand:** Documentación sobre gestión de estado global simplificado. (<https://zustand-demo.pmnd.rs/>)
- **Tailwind CSS & Shadcn UI:** Framework de diseño y componentes accesibles. (<https://ui.shadcn.com/>)
- **Supabase:** Plataforma Backend y arquitectura RLS. (<https://supabase.com/>)
- Apuntes y documentación interna del módulo de Proyecto de Desarrollo de Aplicaciones Web del IES Albarregas.